

# OUPi Robot Sumo: Quick Start

Pololu Zumo 2040 kit (v3). Read this first.

**All of the robot code is public on GitHub.**

<https://github.com/jhassell/oupi-zumo-sumo> has every robot program, the clean baseline, the flashing scripts, and these docs. Clone or download it to set up any laptop: `git clone https://github.com/jhassell/oupi-zumo-sumo`

## 1. How the game works

Two self-driving robots fight in a ring made of **bare wood**. The boundary is the **physical edge of the board, a drop-off with no tape or line**. A robot wins by pushing the other one off the board. Each robot also watches the edge with its downward sensors and tries not to drive itself off.

Players don't steer the robots. Each student designs one by choosing five behavior options (Class, Aggression, Agility, Edge Nerve, Signature) plus a name/color and one free-text "personal tweak." A facilitator running **Claude Code** (on the laptop's command line) turns those choices into real robot code and writes it straight onto the robot over the USB cable. A connected robot appears on the laptop as a writable drive, just like a thumb drive or SD card. Two students' robots then face off head-to-head in the ring, student against student, robot to robot, for a clear winner.

The robots find each other with infrared, including "hearing" the other robot across the ring; the moment one clearly sees an opponent it jumps to full power, and once it is pushing it stays committed. Each robot shows its letter (A/B/C) and team name on its little screen.

## 2. Set it up: the quick version

1. Get the kit from GitHub (clone the repo, or download the ZIP) and start **Claude Code** inside the `oupi-zumo-sumo` folder.
2. Plug a robot into the laptop (it appears as a USB drive named MicroPython).
3. Say **"restart"**: Claude loads the clean baseline and shows the build menu.
4. The student types five numbers (plus an optional name/color and a tweak); Claude builds and flashes the robot.
5. Reset the robot, calibrate if it's a new arena (C → wood → edge), press **A** to fight. Next student → say "restart" again.

## 3. Set it up: step by step

### Get the kit

- Clone the public repo (the repo *is* the kit): `git clone https://github.com/jhassell/oupi-zumo-sumo`. No git? Open the repo page, click the green **Code** button, choose **Download ZIP**, and unzip it.
- Start **Claude Code** inside the resulting `oupi-zumo-sumo` folder so it loads the project rules. Each laptop just clones its own copy; the GitHub repo is the source of truth.
- `sumo2_BASELINE.py` is the clean default (never edited). `custom/sumo2.py` is the working file that students' builds are written into and that gets flashed.
- See which robots are connected and their A/B/C letter: `python3 scripts/identify_bots.py` (identify a robot by its letter, not the drive name).

### The per-student loop

6. Say **"restart"**. Claude resets `custom/sumo2.py` to the baseline, flashes it (ejecting cleanly), and shows the numbered menu.
7. The student replies with one number per row (a blank row uses option 2, the middle):

| Category   | Type 1   | Type 2 (blank = this) | Type 3    |
|------------|----------|-----------------------|-----------|
| Class      | ROOKIE   | STUDENT               | PROFESSOR |
| Aggression | CAUTIOUS | BALANCED              | RECKLESS  |
| Agility    | SLUGGISH | NIMBLE                | TWITCHY   |

|            |         |         |                 |
|------------|---------|---------|-----------------|
| Edge Nerve | CAREFUL | NORMAL  | DAREDEVIL       |
| Signature  | CHARGER | FLANKER | COUNTER-PUNCHER |

...then, if they like, a team name + color and **one short “personal tweak”** in plain English (e.g. “spin on contact,” “victory dance,” “hit at full torque”). Claude implements it in code.

8. Claude applies the choices, checks the code, and flashes: `python3 scripts/flash_robot.py custom/sumo2.py --eject`.

9. Press the robot’s **reset** button (the drive was already ejected), recalibrate if needed, and press **A** to fight. There’s no fixed “house” robot: students compete against each other, robot to robot. Build each student’s robot the same way (one robot connected at a time so each gets its own build), then send two into the ring.

### Calibration (once per robot, per arena)

- Press **C** when stopped. **WOOD SAMPLE:** sit the robot flat on the bare wood, press C. **EDGE SAMPLE:** hold its front sensors out over the board edge (the drop-off), press C.
- It saves the result and reuses it on every reset, so you calibrate once per arena. Press C anytime (when stopped) to redo it if you change boards or the lighting.

### Buttons

| Button | What it does                                 |
|--------|----------------------------------------------|
| A      | Start / stop a match                         |
| B      | Change difficulty (Class), only when stopped |
| C      | Recalibrate the edge, only when stopped      |

### ⚠ Most important rule: always EJECT

**Always eject the MicroPython drive before you remove the USB cable or reset a robot.** On the Linux desktop, right-click the MicroPython disk and choose *Eject* (or the file manager’s eject button, or the flash script’s `--eject`). Pulling the cable before ejecting can corrupt the robot’s memory. If a drive shows 0 bytes or read-only, the robot is running a program, so press reset or replug it to remount, then flash.

### If a robot won’t respond

If a robot isn’t reachable, doesn’t mount, or shows an error on its screen, its operating system probably needs reloading. Copy the entire contents of `bot_image/` (from the repo) onto the robot’s MicroPython drive, eject, and reset. That restores the MicroPython OS, the Pololu library, and every program.

## 4. If your arena is PAINTED (or different)

**Recalibrate first.** This kit’s arena is bare wood with a physical drop-off edge (no border). A painted board reflects the sensors differently, so press **C** (when stopped) and re-sample the new surface and its edge.

**You may also need a code change.** The edge detection currently learns the difference between **bare wood** and the **drop-off at the board’s physical edge**. If the new arena breaks that (a painted surface, a **painted border line** instead of a drop-off, a glossy/reflective finish, a different color, or **no border at all**), calibration alone may not be enough. Describe the arena to Claude Code in plain words: the surface color and finish, and what the boundary is (a painted line and its color/width, just the physical edge, or none). Claude Code can then rewrite the edge-detection and calibration so the robots read the new arena correctly and stay in the ring.

All code & docs: <https://github.com/jhassell/oupi-zumo-sumo> · New build: say “**restart**” to Claude Code